

PRESENTACIÓN
PIGIU

**PLATAFORMA INTEGRAL DE GESTIÓN
DE INCIDENCIAS URBANAS**

Estudiantes:

Mandujano Coronel, Gean Carlos
Lagones Briceno, Will
Salazar Yauri, Javier

Docente: Rosario Delia Osorio Contreras

2026-I · Huancayo, Perú

Que vamos a presentar



Problema y requerimientos

Unidad I y II — que necesita resolver el sistema



Modelos iniciales del sistema

Unidad I y II — casos de uso y modelo de datos



Diseno del software

Unidad III — arquitectura, patrones y base de datos



Interfaz, prototipo y HCI

Unidad IV — usuarios reales y pantallas



Integracion y trazabilidad

Como se verifica que todo este conectado



Trabajo en equipo

Evidencia de colaboracion en GitHub y Jira

Un proyecto alineado al ODS 11



ODS 11 — Ciudades y Comunidades Sostenibles

- Meta 11.3: urbanización inclusiva y planificación participativa de la ciudad.
- Meta 11.6: reducir el impacto ambiental, sobre todo en la gestión de residuos.
- PIGIU conecta a la ciudadanía con la municipalidad para atender estas metas.

Beneficiarios del sistema

- Municipalidad Provincial de Huancayo y municipalidades distritales (El Tambo, Chilca).
- Juntas vecinales y comites comunales, con un canal oficial de reporte.
- Ciudadanía en general, que participa activamente en mejorar su entorno.
- MINAM y organismos fiscalizadores, con datos agregados de gestión ambiental.

Que problema resuelve PIGIU



Hoy no existe un canal formal, digital y trazable para que el ciudadano reporte incidencias urbanas en Huancayo, El Tambo y Chilca.

- Canales informales: llamadas, visitas o redes sociales, sin registro oficial.
- Sin trazabilidad: el ciudadano no recibe confirmacion ni actualizaciones.
- Sin georreferenciacion: no se puede priorizar la atencion por sector.
- Sin metricas de gestion: no hay forma de evaluar el desempeno municipal.
- Duplicacion de esfuerzos: casos repetidos mientras otros quedan sin atender.

62 % de ciudadanos de ciudades intermedias del Peru sienten que sus reportes no reciben atencion oportuna (INDECI, 2023).

Cuatro actores, cuatro necesidades distintas



Ciudadano reportante

Registrar en menos de 60 segundos y conocer el estado de su reporte.



Tecnico operativo

Ver el mapa de su sector y actualizar el caso desde el campo.



Funcionario municipal

Derivar casos a la unidad correcta y generar reportes mensuales.



Alcaldia / Gerencia

Un dashboard ejecutivo con KPIs de desempeño en tiempo real.

Nueve requerimientos en cuatro procesos



1. Registro

RF-01 a RF-03

Incidencia con foto y GPS, categorias y autenticacion.



2. Derivacion

RF-04 y RF-05

Asignacion automatica a la unidad responsable y mapa interactivo.



3. Atencion

RF-06

Ciclo de vida del reporte en seis estados, con responsable y fecha.



4. Seguimiento

RF-07 a RF-10

Reportes estadisticos, notificaciones push y valoracion del servicio.

Ejemplo — RF-01: el sistema genera el ticket en menos de 3 segundos y exige una foto de al menos 640x480 px.

Calidad medible, no solo buenas intenciones

Disponibilidad	99.5% mensual ($\leq$ 3.6 h de caida no planificada)
Rendimiento	Mapa con mas de 1,000 puntos en menos de 3 segundos
Usabilidad	Registro de incidencia en 4 pasos y menos de 60 segundos
Seguridad	Datos cifrados con AES-256 y comunicaciones con TLS 1.3
Privacidad	Cumplimiento de la Ley N.° 29733 de Proteccion de Datos
Escalabilidad	Hasta 10,000 usuarios concurrentes sin degradar el servicio
Portabilidad	Android 8.0+ e iOS 13.0+, el 95% del mercado peruano
Mantenibilidad	Cobertura de pruebas unitarias igual o mayor a 70%

El sistema tambien cumple la ley

Transparencia

Ley N.º 27806

Informe mensual publico de incidencias atendidas.

Gestion de residuos (PIGARS)

Ley N.º 27314

Reportes de basura validados contra rutas de recojo.

Catastro y jurisdiccion

Ley N.º 27972

Verifica que la incidencia este dentro del municipio.

Proteccion de datos (ARCO)

Ley N.º 29733

Acceso, rectificacion, cancelacion y oposicion de datos.

Como se ve el sistema desde afuera



Diagrama de contexto

- PIGIU se trata como una caja negra que recibe y entrega informacion.
- Entran: foto, GPS, descripcion y credenciales de acceso.
- Salen: ticket, notificaciones, reportes y datos de transparencia.
- Se conecta con el Catastro Municipal y con el plan PIGARS.

Cuatro actores, catorce casos de uso

- Ciudadano: registrar, adjuntar evidencia, consultar y reabrir (CU-01 a CU-05).
- Tecnico operativo: consultar, actualizar estado, adjuntar evidencia (CU-06 a CU-08).
- Funcionario municipal: derivar y reasignar reportes (CU-09 y CU-10).
- Administrador / Alcaldia: reportes, usuarios y categorias (CU-11 a CU-14).

Las piezas de informacion y como se conectan



La entidad reporte es el centro del modelo: cada incidencia que reporta un ciudadano queda conectada con su categoria, su evidencia y con quien la atiende.

ciudadano → reporte	1 a muchos: un ciudadano puede tener varios reportes.
categoria → subcategoria	1 a muchos: cada categoria agrupa varias subcategorias.
reporte → evidencia	1 a muchos: un reporte puede tener varias fotos de evidencia.
tecnico / funcionario → reporte	Atienden y supervisan el reporte hasta su cierre.

De los datos a los objetos: clases y secuencias



Diagrama de clases

- Reporte: clase central, con metodos para confirmar, actualizar y derivar.
- Ciudadano: registra incidencias, consulta y valora la atencion.
- Tecnico y Funcionario: atienden, supervisan y reasignan casos.
- Notificacion y Valoracion: cierran el ciclo con el ciudadano.

Tres diagramas de secuencia

- RF-01 Registrar incidencia: app, GPS, camara, API y motor de derivacion.
- Derivacion y atencion: notifica a la unidad, asigna tecnico, marca resuelto.
- Seguimiento y cierre: valoracion del ciudadano y calculo de KPIs.

Scrum y Jira para organizar el trabajo



Con un equipo de 3 integrantes y requisitos que podian cambiar junto con la municipalidad, Scrum permitio entregar valor desde el primer sprint.

Roles del equipo

- Product Owner: prioriza el backlog y valida criterios de aceptacion.
- Scrum Master: facilita ceremonias y elimina impedimentos.
- Developer Team: backend, frontend/movil y QA / DevOps.

Siete sprints, de la base al despliegue

- Sprint 0–1: arquitectura base y registro de incidencias (MVP).
- Sprint 2–3: derivacion automatica y app del tecnico offline.
- Sprint 4–6: dashboard, transparencia, pruebas e implementacion.
- Velocidad de referencia: 26 puntos de historia por sprint.

Arquitectura en cuatro capas



1. Presentacion

App ciudadana, app del tecnico y panel web (React Native / React.js).



2. Logica de negocio

API REST en Node.js y Express: reglas, derivacion y eventos.



3. Acceso a datos

Patron DAO sobre PostgreSQL + PostGIS, con Sequelize.



4. Infraestructura

AWS S3, Firebase, mapas y Redis para cache y eventos.

Arquitectura en capas + orientada a eventos para el mapa y las notificaciones en tiempo real.

Cuatro patrones, cuatro problemas resueltos



DAO

Separa la base de datos de la logica de negocio y permite probar con datos simulados.



Observer

Un cambio de estado avisa a la vez al ciudadano, al mapa y al panel de KPIs.



Factory Method

Cada categoria de incidencia tiene su propio manejador de derivacion.



Singleton

Una sola conexion compartida a la base de datos y a las notificaciones.

Una base de datos normalizada y geoespacial



PostgreSQL + PostGIS

- Normalizada hasta tercera forma normal (3FN), sin datos repetidos.
- Ejemplo: las fotos pasaron de un campo de texto a la tabla evidencias.
- Índice geoespacial GIST para ubicar incidencias en el mapa al instante.
- Tabla historial_estados: trazabilidad completa de cada cambio.

Seguridad y respaldo de datos

- Control de acceso por roles (RBAC): cada usuario ve solo lo necesario.
- Respaldo completo semanal, diferencial diario y registro continuo (WAL).
- Replica en otra zona con recuperación automática en menos de 1 minuto.
- Cumple el RNF-01 de disponibilidad del 99.5% mensual.

Tres canales para tres necesidades distintas



API REST

Operaciones transaccionales: registro, consulta, autenticacion.



WebSocket

Actualiza el mapa del panel administrativo sin recargar la pagina.



Firebase (FCM)

Notificaciones push al ciudadano y al tecnico, incluso en segundo plano.

Modo offline del tecnico operativo

- El tecnico trabaja en zonas con senal intermitente; la app detecta la caida en menos de 2 segundos.
- Las acciones se guardan en una base local (SQLite) y se sincronizan al recuperar la conexion.
- Si hay un conflicto de version, la app muestra ambos estados para que el tecnico decida.

Seguridad en cada capa del sistema



La seguridad no se agrega al final: esta presente desde la autenticacion hasta el almacenamiento de cada dato.

- Autenticacion: JWT y OAuth para ciudadanos; LDAP institucional para funcionarios.
- Cifrado: AES-256 para datos en reposo y TLS 1.3 para datos en transito.
- Proteccion contra fuerza bruta: bloqueo tras 5 intentos fallidos de acceso.
- Prevencion de inyeccion SQL mediante consultas preparadas (Sequelize ORM).
- Cabeceras de seguridad HTTP (Helmet.js) frente a ataques comunes del navegador.
- Auditoria realizada con la guia OWASP durante el Sprint 6 de pruebas.

Disenar para cuatro personas reales



Maria, 42 anos

Ciudadana. Necesita registrar en pocos pasos, sin tecnicismos.



Carlos, 31 anos

Tecnico de campo. Necesita botones grandes y modo offline.



Jorge, 47 anos

Funcionario. Necesita el mapa como pantalla principal.



Dra. Rosa, 55 anos

Alcaldesa. Necesita KPIs claros en menos de 30 segundos.

Seis heurísticas de Nielsen en el prototipo



Consistencia

Un mismo color y texto de estado en las tres plataformas.



Visibilidad del estado

Barra de progreso e indicador de conexión siempre visibles.



Accesibilidad

Contraste mínimo 4.5:1 y botones de al menos 48 dp.



Control del usuario

El ciudadano puede retroceder sin perder lo que escribió.



Retroalimentación

Confirmación visual inmediata tras cada acción importante.



Simplicidad

Solo 5 campos obligatorios en el registro inicial.

Del wireframe a la pantalla final



Proceso iterativo

- Wireframes de baja fidelidad probados con 5 ciudadanos y 2 tecnicos.
- Hallazgo: el icono de ubicacion no se entendia sin una etiqueta de texto.
- Hallazgo: el boton de 'Resuelto' quedaba oculto al final de la pantalla.
- Ambos hallazgos se corrigieron antes de programar una sola linea de codigo.

16 pantallas de alta fidelidad

- Diseñadas en Figma, para app ciudadana, app tecnico y panel web.
- Sistema de diseno comun: color azul institucional, tipografia Roboto.
- Espaciado en multiplos de 8 dp, igual en Figma y en el codigo.
- Color de exito, advertencia y error siempre acompañado de un icono.

Cada pantalla responde a un requerimiento

- App ciudadana: inicio, asistente de nuevo reporte en 4 pasos, notificaciones y perfil.
- Panel web: mapa, bandeja de incidencias, reportes, dashboard ejecutivo y administracion.

12 pantallas principales cubren **9 requerimientos funcionales**, los requerimientos de dominio relacionados con la interfaz y los criterios de usabilidad y seguridad.

Todo requerimiento, verificado

95.7%

22 de 23 requerimientos cumplidos en
el diseno actual

- RF ↔ caso de uso ↔ clase o servicio ↔ pantalla del prototipo.
- RNF ↔ mecanismo tecnico ↔ evidencia de prueba (carga, seguridad).
- RD ↔ norma legal ↔ modulo responsable del sistema.
- El unico item parcial es RD-02 (PIGARS): depende de un convenio institucional aun pendiente con la municipalidad; mientras tanto se usa una capa GeoJSON del IGN.

Arquitectura, base de datos y prototipo: una sola historia



Arquitectura → Prototipo

El patron Observer hace que un cambio de estado se vea de inmediato en las 4 notificaciones push documentadas en el prototipo.



Arquitectura → Base de datos

El campo de control de version (conurrencia) tiene su propia pantalla de resolucion de conflicto en la app del tecnico.



Revision heuristica

La revision con las heurísticas de Nielsen encontro 5 errores de usabilidad y se corrigieron antes de programar.

Evidencia real de trabajo colegiado



Organizacion del repositorio

- Estrategia GitFlow: ramas main, develop, feature, release y hotfix.
- Integracion con Jira mediante Smart Commits, ligando codigo con historias de usuario.
- Mas de 130 commits y 30 Pull Requests revisados entre los tres integrantes.

Aprendizajes del equipo

- Se adopto la convencion Conventional Commits tras notar mensajes poco claros.
- Se protegio la rama main exigiendo revision de un companero antes de fusionar.
- El 80% de los errores de integracion se detecto en la revision de codigo, antes de llegar a pruebas.

Conclusiones y recomendaciones



Conclusiones

- PIGIU digitaliza todo el ciclo de la incidencia: registro, derivacion, atencion y cierre.
- El problema no era solo tecnologico: tambien faltaban procesos internos definidos.
- Scrum permitio entregar valor de forma incremental, sprint a sprint.
- El sistema cumple, de forma directa, la normativa peruana revisada.



Recomendaciones

- Iniciar con un piloto controlado en El Tambo antes del despliegue completo.
- Diseñar capacitacion diferenciada para tecnicos, funcionarios y ciudadania.
- Integrar a futuro con la Plataforma Digital del Estado Peruano (GOB.PE).
- Incorporar clasificacion automatica de fotos con inteligencia artificial.

REFERENCIAS



Espacio

Proyecto PIGIU

Resumen

Backlog

Tablero

Código

Cronograma

Buscar en el ba...

GC JY NS WB

Filtro

El ciudadano puede registrarse, iniciar sesión y registrar una incidencia con foto y GPS, recibiendo su número de ticket de confirmación. Al finalizar el sprint, se tiene un flujo de reporte funcional de extremo...

☒	SCRUM - 1	Registro de usuario ciudadano (correo / Google)		REGISTRO DE INCIDE...	EN CURSO	1 jul	5		JY
☐	SCRUM - 2	Captura fotográfica de incidencia con validación de calidad		REGISTRO DE INCIDE...	EN CURSO	1 jul	8		GC
☒	SCRUM - 7	Geolocalización automática GPS con ajuste manual en mapa		REGISTRO DE INCIDE...	EN CURSO	2 jul	8		WB
☒	SCRUM - 8	Clasificación de incidencia por categoría y subcategoría		REGISTRO DE INCIDE...	EN CURSO	2 jul	5		GC
☒	SCRUM - 9	Generación de ticket único y confirmación al ciudadano		REGISTRO DE INCIDE...	EN CURSO	3 jul	3		WB

+ Crear

☐ SPRINT 2 – Derivación Automáti

26 may – 3 jun (4 actividades)

Los funcionarios municipales pueden visualizar incidencias en el mapa interactivo, recibir casos derivados automáticamente y gestionar el ciclo de vida de las incidencias.

☒	SCRUM - 10	Derivación automática de incidencias a unidad responsable		DERIVACIÓN DE CASOS	EN
☒	SCRUM - 11	Mapa interactivo de incidencias para panel administrativo		DERIVACIÓN DE CASOS	EN
☒	SCRUM - 12	Reasignación manual de casos con bitácora de auditoría		DERIVACIÓN DE CASOS	EN

Pulse

Contributors

Community

Community standards

Traffic

Commits

Code frequency

Dependency graph

Network

Forks

Actions usage metrics

Actions performance metrics

June 16, 2026 – June 23, 2026

Period: 1 week

Overview

0 Active pull requests

0 Active issues

0 Merged pull requests

0 Open pull requests

0 Closed issues

0 New issues

Summary

Excluding merges, 3 authors have pushed 7 commits to main and 7 commits to all branches.

On main, 5 files have changed and there have been 2 additions and 2 deletions

Top committers

Commits

4

2

2

0

Avatar 1

Avatar 2

Avatar 3

Gracias

PIGIU — Construyendo un canal trazable entre la ciudadanía y la gestión municipal.



<https://www.figma.com/make/jX52MrgHZi51WLCLFIhXTF/Expand-interactive-examples?fullscreen=1&t=6nG5zY3S7avq42Q8-1&code-node-id=0-9> *(figma VF)*



github.com/Gean77958/pigiui-proyecto *(git hub)*



<https://continental-team-bpfrsx4g.atlassian.net/jira/software/projects/SCRUM/boards/1/backlog?selectedIssue=SCRUM-166> *(JIRA)*

